CPSC-406 Report

Farzana Chowdhury
Chapman University

April 9, 2026

Abstract

Contents

1 Introduction **2**

2 Week by Week **2**
2.1 Week 1 . . . 2
2.1.1 DFAs . . . 2
2.1.2 Question . . . 4
2.2 Week 2 . . . 4
2.2.1 Operations on automata . . . 4
2.2.2 Question . . . 6
2.3 Week 3 . . . 6
2.3.1 NFAs vs DFAs and Determinization . . . 6
2.3.2 Question . . . 8
2.4 Week 4 . . . 8
2.4.1 HW 4 . . . 8
2.4.2 Question . . . 10
2.5 Week 5 . . . 10
2.5.1 Problems on the Pumping Lemma . . . 10
2.5.2 Question . . . 11
2.6 Week 6 . . . 11
2.6.1 HW 6 . . . 11
2.6.2 Question . . . 12
2.7 Week 7 . . . 12
2.7.1 HW 7 . . . 12
2.7.2 Question . . . 13
2.8 Week 8 . . . 13
2.8.1 Propositional Logic and SAT . . . 13
2.8.2 Question . . . 14

3 Synthesis **14**

4 Evidence of Participation **14**

5 Conclusion **14**

1 Introduction

2 Week by Week

2.1 Week 1

2.1.1 DFAs

Exercise 1

1. Which of the following words are accepted/refused by $\mathcal{A}_1$ and $\mathcal{A}_2$? Complete the table.

:w	**accepted by $\mathcal{A}_1$?**	**accepted by $\mathcal{A}_2$?**
aaa	$\times$	✓
aab	✓	$\times$
aba	$\times$	$\times$
abb	$\times$	$\times$
baa	$\times$	✓
bab	$\times$	$\times$
bba	$\times$	$\times$
bbb	$\times$	$\times$

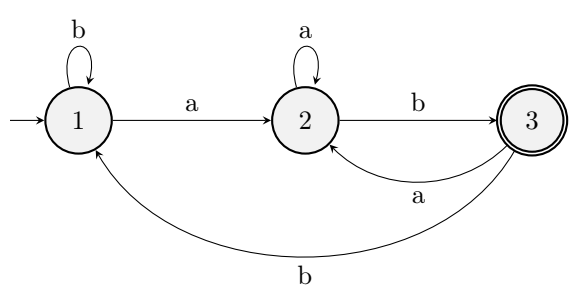
2. More generally, can you completely describe the languages $L(\mathcal{A}_k)$ accepted by $\mathcal{A}_k$, for $k = 1, 2$?

- $L(\mathcal{A}_1)$: The language of all words over $\{a, b\}$ that start with the letter a and end with an odd number of consecutive b's.
- $L(\mathcal{A}_2)$: The language of all words over $\{a, b\}$ that end with the substring aa.

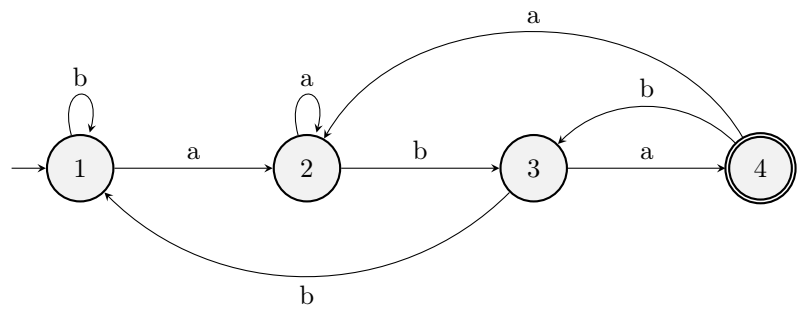
Exercise 2

Design DFAs whose accepted languages are given as follows:

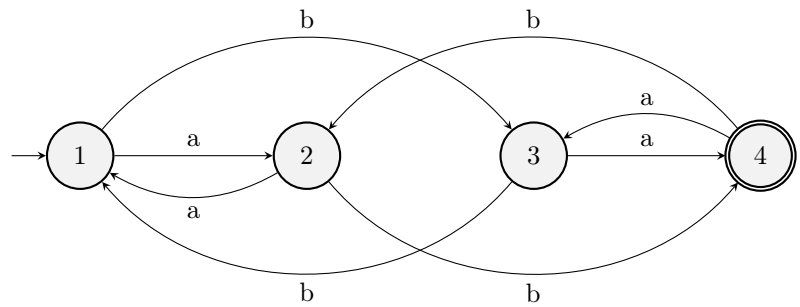
1. All the words that end with ab



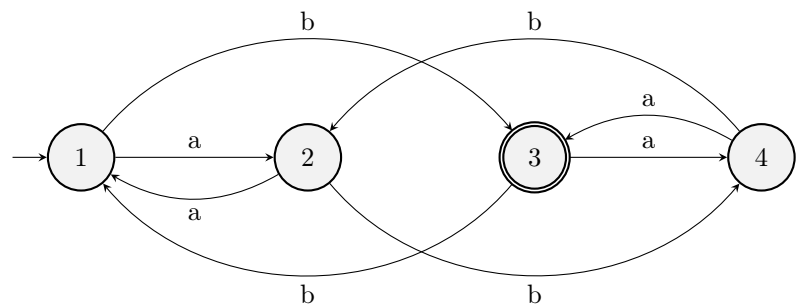
2. All the words that contain aba



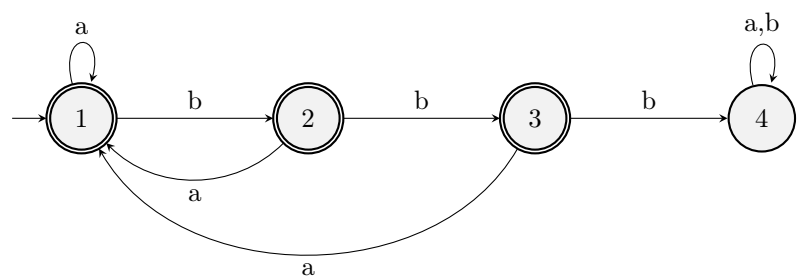
3. All the words that contain an odd number of a's and an odd number of b's



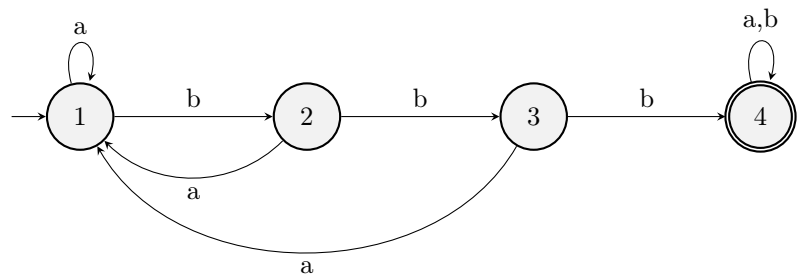
4. All the words that contain an even number of a's and an odd number of b's



5. All the words such that any three consecutive characters contain at least one a



6. All the words that contain bbb



What do you notice when comparing the various automata?

For DFAs that have criteria of "ends with," arrows must leave the accept state because the string could keep going and fail to satisfy DFA characteristics. For DFAs that have criteria of "contains," the accept state must be a "Trap State" with a self-loop, because once you find the sequence, you can traverse through the DFA while satisfying DFA's accepted language conditions.

2.1.2 Question

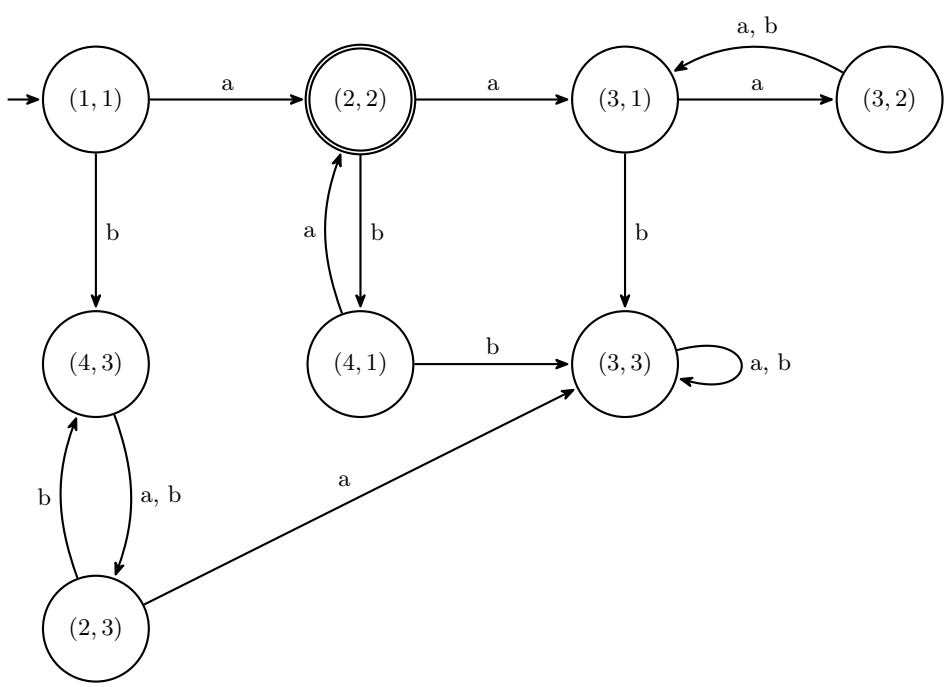
What is the easiest and best way to implement DFAs in programming?

2.2 Week 2

2.2.1 Operations on automata

Exercise 1

1. $L(\mathcal{A}^{(1)})$ can't have aa or bb (two of the same letters in a row). For $L(\mathcal{A}^{(1)})$, any a has to be in an odd numbered state position.



2.

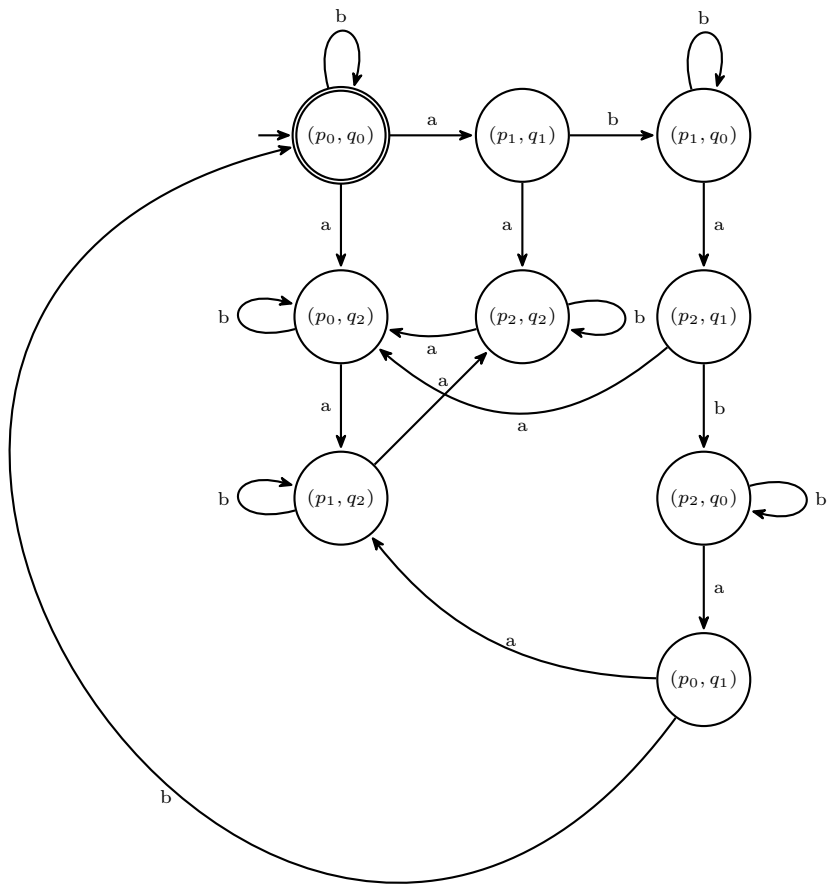
3. To find the intersection of two languages ($L_1 \cap L_2$), you create a new machine that simulates how $\mathcal{A}^{(1)}$ and $\mathcal{A}^{(2)}$ work at the same time. When you take in an input a or b, both numbers in the pair update

at the same time based on their original rules. For intersection ($\cap$), you only have a double circle if both machines are in an accepting state.

4. For union ($\cup$), you have a double circle if at least one of the machines is in an accepting state.

Exercise 2

1. For $L(\mathcal{B}^{(1)})$, the total number of a is a multiple of 3, including zero. For $L(\mathcal{B}^{(2)})$, every a is directly followed by at least one b, and the string must not end with a.



2.

3. Since the only accepting state is (p_0, q_0), a string is accepted if and only if it leads both $L(\mathcal{B}^{(1)})$ and $L(\mathcal{B}^{(2)})$ to an accepting state.

4. A state will be accepting if the first part is p_0 or if the second part is not q_0.

ITALC, Exercise 2.2.7 Basis:
Let the length of the string be $n = 0$. The empty string, ϵ, is of length 0.

$$\hat{\delta}(q, \epsilon) = q$$

$\therefore$ The basis is true.

Inductive Hypothesis:
Assume that for any string w of length n, $\hat{\delta}(q, w) = q$.

Inductive Step:
Consider a string x of length $n + 1$. The string can be written as $x = wa$, where w is a string of length n and a is a single input symbol ($a \in \Sigma$).

$$\begin{aligned}
\hat{\delta}(q, wa) &= \delta(\hat{\delta}(q, w), a) && \text{– by definition of } \hat{\delta} \\
&= \delta(q, a) && \text{– by Inductive Hypothesis: } \hat{\delta}(q, w) = q \\
&= q && \text{– by given property: } \delta(q, a) = q \text{ for all } a
\end{aligned}$$

2.2.2 Question

What other mathematical properties aside from intersection and union could be applied to automata?

2.3 Week 3

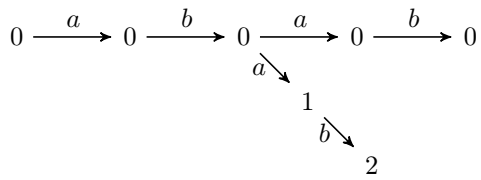
2.3.1 NFAs vs DFAs and Determinization

Homework 1

1. **Regular Expression:** $L(\mathcal{A}) = \{(a|b)^* a(b|c)\}$
2. **NFA Definition:**
   - $Q = \{0, 1, 2, 3\}$
   - $\Sigma = \{a, b, c\}$
   - δ (Transition Function):

State	a	b	c
0	$\{0, 1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\{3\}$
2	$\emptyset$	$\emptyset$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

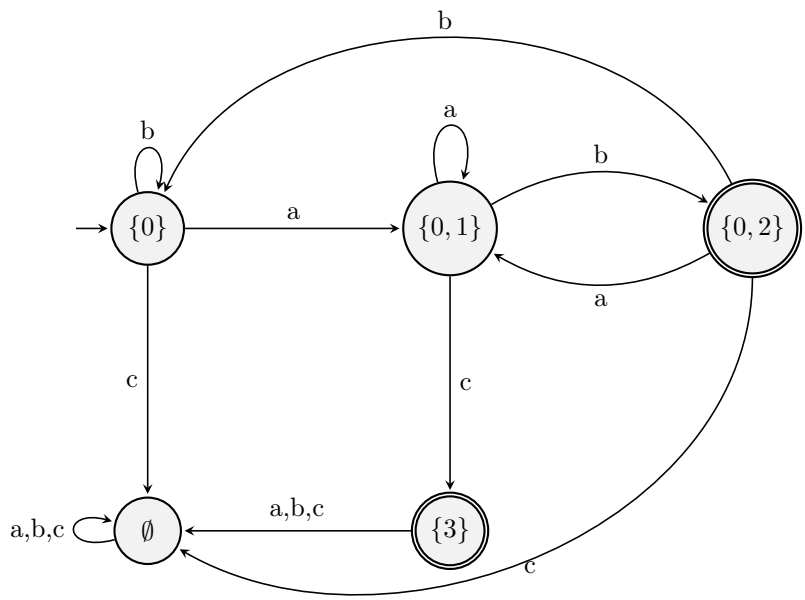
   - $q_0 : 0$ (Initial State)
   - $F : \{2, 3\}$ (Final States)
3. **Paths in $\mathcal{A}$ for the word $v = abab$:**



4. **Determinization $\mathcal{A}^{\mathrm{D}}$ of $\mathcal{A}$ Transition Table Using the Power Set Construction:**

States	a	b	c
$\rightarrow \{0\}$	$\{0,1\}$	$\{0\}$	$\emptyset$
$\{0,1\}$	$\{0,1\}$	$\{0,2\}$	$\{3\}$
$\{0,2\}$	$\{0,1\}$	$\{0\}$	$\emptyset$
$\{3\}$	$\emptyset$	$\emptyset$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Determinization $\mathcal{A}^{\mathrm{D}}$ of $\mathcal{A}$ Diagram Using the Power Set Construction:

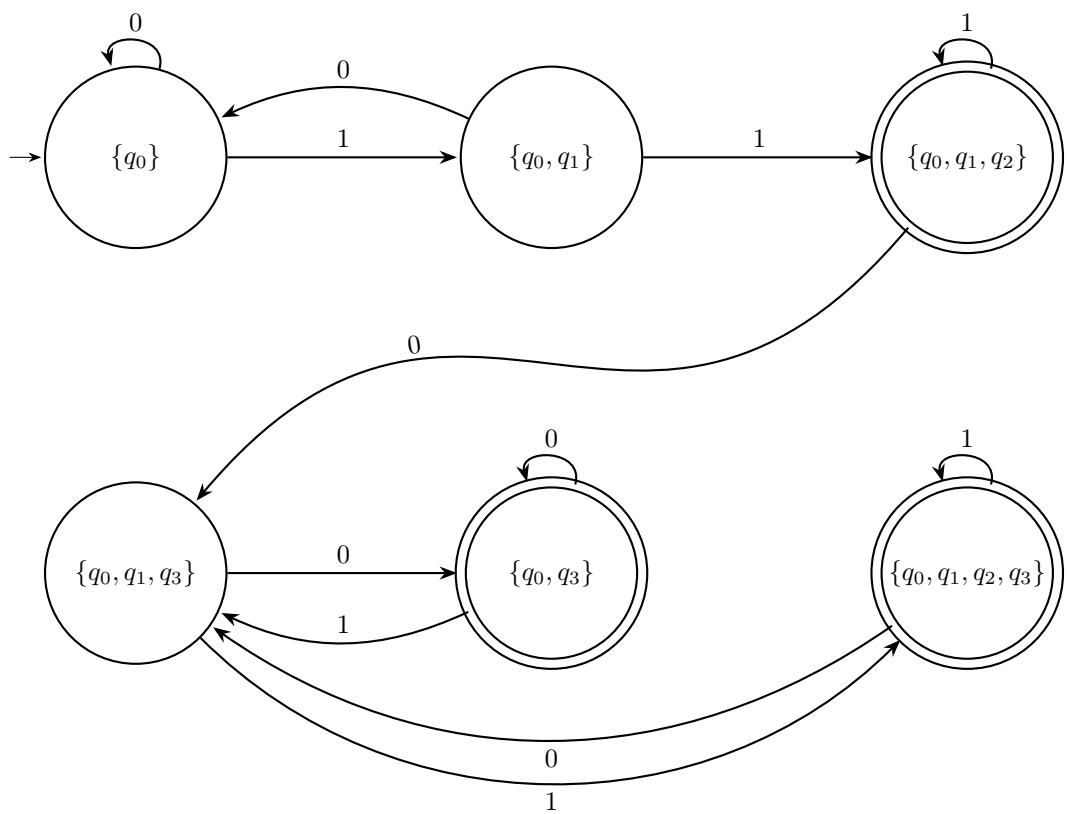


Homework 2

1. **Determinization $\mathcal{A}^{\mathrm{D}}$ of $\mathcal{A}$ Transition Table Using the Power Set Construction:**

States	0	1
$\rightarrow \{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_1, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$
$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_1, q_2, q_3\}$

Determinization $\mathcal{A}^{\mathrm{D}}$ of $\mathcal{A}$ Diagram Using the Power Set Construction:



2. There is a smaller DFA where $\{q_0, q_1, q_3\}$, $\{q_0, q_3\}$, $\{q_0, q_1, q_2, q_3\}$ will be merged into one accepting state. This is due to all of them leading back to each other despite the input; they are essentially equivalent.

2.3.2 Question

Do you always have to change NFAs to be DFAs when programming?

2.4 Week 4

2.4.1 HW 4

Exercise (Kleene's Algorithm)

$k = 0$:

$$\begin{aligned} R_{11}^{(0)} &= \varepsilon + a \\ R_{12}^{(0)} &= b \\ R_{21}^{(0)} &= a \\ R_{22}^{(0)} &= \varepsilon + b \end{aligned}$$

$k = 1$:

$$\begin{aligned} R_{11}^{(1)} &= R_{11}^{(0)} + R_{11}^{(0)}(R_{11}^{(0)})^* R_{11}^{(0)} \\ &= (\varepsilon + a) + (\varepsilon + a)(\varepsilon + a)^*(\varepsilon + a) \\ &= a^* \end{aligned}$$

$$\begin{aligned} R_{12}^{(1)} &= R_{12}^{(0)} + R_{11}^{(0)}(R_{11}^{(0)})^* R_{12}^{(0)} \\ &= b + (\varepsilon + a)(\varepsilon + a)^* b \\ &= a^* b \end{aligned}$$

$$\begin{aligned} R_{21}^{(1)} &= R_{21}^{(0)} + R_{21}^{(0)}(R_{11}^{(0)})^* R_{11}^{(0)} \\ &= a + a(\varepsilon + a)^*(\varepsilon + a) \\ &= aa^* \end{aligned}$$

$$\begin{aligned} R_{22}^{(1)} &= R_{22}^{(0)} + R_{21}^{(0)}(R_{11}^{(0)})^* R_{12}^{(0)} \\ &= (\varepsilon + b) + a(\varepsilon + a)^* b \\ &= \varepsilon + b + aa^* b \end{aligned}$$

$k = 2$:

$$\begin{aligned} R_{11}^{(2)} &= R_{11}^{(1)} + R_{12}^{(1)}(R_{22}^{(1)})^* R_{21}^{(1)} \\ &= a^* + (a^* b)(\varepsilon + b + aa^* b)^*(aa^*) \\ &= a^* + (a^* b)(b + aa^* b)^*(aa^*) \end{aligned}$$

Final Regular Expression:

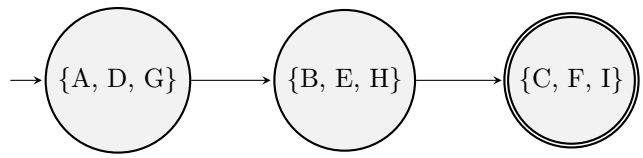
$$R = a^* + a^* b(b + aa^* b)^* aa^*$$

ITALC, Exercise 4.4.2.

a. The state minimization table:

	A	B	C	D	E	F	G	H
B	X							
C	X	X						
D		X	X					
E	X		X	X				
F	X	X		X	X			
G		X	X		X	X		
H	X		X	X		X	X	
I	X	X		X	X		X	X

b. The minimized DFA diagram:



2.4.2 Question

Where would an implementation of Kleene's Algorithm be the most beneficial?

2.5 Week 5

2.5.1 Problems on the Pumping Lemma

Exercise

$L_1 = \{a^n b^m \in \{a,b\}^* \mid n, m \in \mathbb{N},\ n > m \geq 0\}$

- **String Choice:** Let $s = a^{p+1}b^p$. Note $s \in L_1$ because $p+1 > p$ and $|s| \geq p$.
- **Decomposition:** $s = xyz$ such that $|xy| \leq p$ and $|y| > 0$. Since the first p characters are all a's, $y = a^k$ for some $1 \leq k \leq p$.
- **Pumping:** Consider $i = 0$ (pumping down). The resulting string is $s' = xy^0z = xz$.
- **Contradiction:** The number of a's in s' is $(p+1)-k$. Since $k \geq 1$, $(p+1)-k \leq p$. The number of b's remains p. Thus, the number of a's is no longer strictly greater than the number of b's ($p-k+1 \not> p$), so $s' \notin L_1$.

$L_2 = \{a^{n^2} \in \{a\}^* \mid n \in \mathbb{N},\ n \geq 1\}$

- **String Choice:** Let $s = a^{p^2}$. Note $s \in L_2$ and $|s| = p^2 \geq p$.
- **Decomposition:** $y = a^k$ where $1 \leq k \leq p$.
- **Pumping:** Consider $i = 2$ (pumping up). The length of xy^2z is $|s| + |y| = p^2 + k$.
- **Contradiction:** We know that $p^2 < p^2 + k$. Since $k \leq p$, we also know:

$$p^2 + k < p^2 + 2p + 1 = (p+1)^2$$

  Because $p^2 < |xy^2z| < (p+1)^2$, the length is trapped between two consecutive perfect squares and cannot be a perfect square itself. Thus, $xy^2z \notin L_2$.

$L_3 = \{a^p \in \{a\}^* \mid p \in \mathbb{N},\ p \text{ prime}\}$

- **String Choice:** Let n be a prime such that $n > p$. Let $s = a^n$.
- **Decomposition:** $y = a^k$ where $1 \leq k \leq p$. The length of s is n.
- **Pumping:** Let $i = n+1$. The length of $xy^{n+1}z$ is:

$$|s| + n|y| = n + nk = n(1+k)$$

- **Contradiction:** Since $n > 1$ and $(1+k) > 1$, the length $n(1+k)$ is a composite number. A string of composite length is not in L_3, so L_3 is not regular.

$L_4 = \{a^k b^m a^{k+m} \mid k, m \geq 0\}$

- **String Choice:** Let $s = a^p b^p a^{2p}$. Note $s \in L_4$ because $p + p = 2p$.

- **Decomposition:** Since $|xy| \leq p$, y must consist only of a's from the first block. Thus $y = a^j$ for $1 \leq j \leq p$.
- **Pumping:** Let $i = 2$. The new string is $s' = a^{p+j}b^p a^{2p}$.
- **Contradiction:** For $s' \in L_4$, the sum of the exponents of the first two blocks must equal the exponent of the last block. However, $(p + j) + p = 2p + j$. Since $j \geq 1$, $2p + j \neq 2p$. Thus $s' \notin L_4$.

2.5.2 Question

Is the pumping lemma specific only to computer science, or could it be applicable to other fields?

2.6 Week 6

2.6.1 HW 6

Turing Machines: Exercise A

1. **TM for $10^n \to 10^{n+1}$:**

State	Read	Write	Move	Next
q_0	1	1	R	q_1
q_1	0	0	R	q_1
q_1	B	0	R	q_{acc}

2. **TM for $10^n \to 1$:**

State	Read	Write	Move	Next
q_0	1	1	R	q_1
q_1	0	0	R	q_1
q_1	B	B	L	q_2
q_2	0	B	L	q_2
q_2	1	1	R	q_{acc}

3. **TM for Swapping 0's and 1's:**

State	Read	Write	Move	Next
q_0	1	0	R	q_0
q_0	0	1	R	q_0
q_0	B	B	R	q_{acc}

Problems on Decidability: Exercise 1

1. **True.** Since L_1 and L_2 are decidable, there exist TMs M_1, M_2 that halt on all inputs. A TM for $L_1 \cup L_2$ runs M_1 then M_2; it accepts if either accepts and rejects if both reject.
2. **True.** If M decides L, we construct M' to decide $\overline{L}$ by swapping the q_{accept} and q_{reject} states of M.
3. **True.** Decidable languages are closed under Kleene star. We can use a dynamic programming approach to check if any prefix $w[1 \ldots i] \in L$ and the remainder $w[i + 1 \ldots n] \in L^*$.
4. **True.** To recognize $L_1 \cup L_2$, run TMs M_1 and M_2 in parallel (interleaving steps). If either halts and accepts, the union TM accepts.
5. **False.** The language A_{TM} (the Halting Problem) is r.e., but its complement $\overline{A_{TM}}$ is not r.e.
6. **True.** We can construct a non-deterministic TM that non-deterministically branches to split the input into k substrings and verifies each substring is in L.

2.6.2 Question

Are there concepts similar to Turing Machines in other fields or are Turing Machines unique to Computer Science?

2.7 Week 7

2.7.1 HW 7

Problems on Decidability: Exercise 2

1. $L_1 := \{M \mid M$ **halts on itself**$\}$
   - **Recursively Enumerable (r.e.): Yes.** We can build a Universal Turing Machine that takes the description of M and simulates M using its own description $\langle M \rangle$ as input. If M halts, the simulation will eventually halt and accept.
   - **Decidable: No.** This is proven via a reduction from the standard Halting Problem. If we could decide L_1, we could decide if any machine M halts on any input w by constructing a new machine M' that ignores its own input and simply runs $M(w)$.
   - **Co-r.e.: No.** If L_1 were co-r.e., and since it is already r.e., it would necessarily be decidable (by running both the r.e. and co-r.e. procedures in parallel). Since it is undecidable, it cannot be co-r.e.
2. $L_2 := \{(M, w) \mid M$ **halts on the word** $w\}$
   - **Recursively Enumerable (r.e.): Yes.** A Universal Turing Machine can simulate M on w. If M halts on w, the simulator halts and accepts.
   - **Decidable: No.** This is the standard Halting Problem (ATM or $HALT_{TM}$). Alan Turing famously proved that no general algorithm exists to decide this for all possible program-input pairs.
   - **Co-r.e.: No.** Similar to L_1, if L_2 were co-r.e., it would be decidable. The complement (detecting if a machine loops forever) is not r.e.
3. $L_3 := \{(M, w, k) \mid M$ **halts on** w **in at most** k **steps**$\}$
   - **Decidable: Yes.** This is the "bounded" version of the halting problem. Unlike the previous two, it is decidable because the search space is finite.
   - **Argument:** To decide L_3, simply simulate M on w for exactly k steps:
     - If the machine enters a halting state at or before step k, **Accept**.
     - If the machine has not halted after k steps, **Reject**.
   - **R.e. and Co-r.e.: Yes.** Since every decidable language is both r.e. and co-r.e. by definition, L_3 fits into all three categories.

Landau Notation: Exercise 2

1. **Reflexive Property:** $f \in \mathcal{O}(f)$
   Proof: Choose $C = 1$ and $n_0 = 1$. For all $n \geq 1$, it is always true that $f(n) \leq 1 \cdot f(n)$. Since the condition holds, $f \in \mathcal{O}(f)$.
2. **Constant Multiples: If** $c > 0$ **is a constant, then** $\mathcal{O}(c \cdot f) = \mathcal{O}(f)$
   We must show both inclusions:

- ($\subseteq$): Let $g \in \mathcal{O}(c \cdot f)$. Then $g(n) \leq C \cdot (c \cdot f(n))$ for $n \geq n_0$. By setting a new constant $C' = C \cdot c$, we get $g(n) \leq C' \cdot f(n)$, so $g \in \mathcal{O}(f)$.
- ($\supseteq$): Let $g \in \mathcal{O}(f)$. Then $g(n) \leq C \cdot f(n)$ for $n \geq n_0$. We can rewrite this as $g(n) \leq \frac{C}{c} \cdot (c \cdot f(n))$. Since $c > 0$, $\frac{C}{c}$ is a valid constant, so $g \in \mathcal{O}(c \cdot f)$.

3. **Dominance: If $f \leq g$ for large enough inputs, then $\mathcal{O}(f) \subseteq \mathcal{O}(g)$**
   Proof: Let $h \in \mathcal{O}(f)$. This means $h(n) \leq C \cdot f(n)$ for $n \geq n_1$. We are given $f(n) \leq g(n)$ for $n \geq n_2$. For all $n \geq \max(n_1, n_2)$, we have $h(n) \leq C \cdot f(n) \leq C \cdot g(n)$. Thus, $h \in \mathcal{O}(g)$.
4. **Additive Property: If $\mathcal{O}(f) \subseteq \mathcal{O}(g)$, then $\mathcal{O}(f + h) \subseteq \mathcal{O}(g + h)$**
   Proof: The premise $\mathcal{O}(f) \subseteq \mathcal{O}(g)$ implies $f \in \mathcal{O}(g)$, so $f(n) \leq C \cdot g(n)$ for $n \geq n_0$. Take any $t \in \mathcal{O}(f+h)$. Then $t(n) \leq C_1(f(n) + h(n))$ for $n \geq n_1$. Substitute the inequality for f: $t(n) \leq C_1(C \cdot g(n) + h(n))$. Let $C_2 = \max(C, 1)$. Then $t(n) \leq C_1 \cdot C_2(g(n) + h(n))$. Since t is bounded by a constant times $(g+h)$, $t \in \mathcal{O}(g + h)$.
5. **Multiplicative Property: If $h(n) > 0$ and $\mathcal{O}(f) \subseteq \mathcal{O}(g)$, then $\mathcal{O}(f \cdot h) \subseteq \mathcal{O}(g \cdot h)$**
   Proof: From the premise, $f(n) \leq C \cdot g(n)$ for $n \geq n_0$. Let $t \in \mathcal{O}(f \cdot h)$, meaning $t(n) \leq C_1(f(n) \cdot h(n))$ for $n \geq n_1$. Multiply the first inequality by $h(n)$. Since $h(n) > 0$, the direction of the inequality remains the same: $f(n) \cdot h(n) \leq C \cdot g(n) \cdot h(n)$. Therefore, $t(n) \leq C_1(C \cdot g(n) \cdot h(n))$. Using the new constant $C_2 = C_1 \cdot C$, we find $t \in \mathcal{O}(g \cdot h)$.

2.7.2 Question

How can we apply Big O algorithms to DFAs, if possible?

2.8 Week 8

2.8.1 Propositional Logic and SAT

Exercise 1

1. **Formula φ_1:**
$$\varphi_1 := \neg((a \wedge b) \vee (\neg c \wedge d))$$
   Conversion:
$$\begin{aligned} \varphi_1 &\equiv \neg(a \wedge b) \wedge \neg(\neg c \wedge d) && \text{(De Morgan's)} \\ &\equiv (\neg a \vee \neg b) \wedge (c \vee \neg d) && \text{(CNF Result)} \end{aligned}$$
2. **Formula φ_2:**
$$\varphi_2 := \neg((p \vee q) \rightarrow (r \wedge \neg s))$$
   Conversion:
$$\begin{aligned} \varphi_2 &\equiv \neg(\neg(p \vee q) \vee (r \wedge \neg s)) && \text{(Implication Rule)} \\ &\equiv \neg(\neg(p \vee q)) \wedge \neg(r \wedge \neg s) && \text{(De Morgan's)} \\ &\equiv (p \vee q) \wedge (\neg r \vee s) && \text{(CNF Result)} \end{aligned}$$

Exercise 2

1. $\psi_1 := (a \vee \neg b) \wedge (\neg a \vee b) \wedge (\neg a \vee \neg b)$
   Status: Satisfiable
   Assignment: $\{a = \text{False}, b = \text{False}\}$

2. $\psi_2 := (\neg p \vee q) \wedge (\neg q \vee r) \wedge \neg(\neg p \vee r)$
   Status: Unsatisfiable
   Explanation: The requirements contradict each other. p makes it so q has to be true, but by q being true, the second clause fails because r must be false.

3. $\psi_3 := (x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y) \wedge (\neg x \vee \neg y)$
   Status: Unsatisfiable
   Explanation: The formula represents all four possible combinations of these two variables, x and y. The second two clauses have the exact opposite requirements of the first two clauses.

2.8.2 Question

Where would propositional logic and SAT be implemented so that it is helpful in simplifying a more complex topic?

3 Synthesis

4 Evidence of Participation

5 Conclusion

References

[BLA] Author, Title, Publisher, Year.